

2022-2023 Final Exam

CPT104 Operating Systems

Exam Answers & Explanations

Generated by OClaw

Question 1 (44 marks)

Q: [4 marks] 1. Can priority scheduling cause starvation? How to fix it?

YES, priority scheduling can cause starvation. Low-priority processes may never get CPU if high-priority processes keep arriving.

Fix: Aging - gradually increase the priority of waiting processes over time so they eventually get executed.

[CHN] 是的，优先级调度可能导致饥饿。低优先级进程可能永远无法获得CPU。

解决方法：老化——逐渐提高等待进程的优先级，使其最终能被执行。

Q: [12 marks] 2. Discuss positive and negative effects of increasing page size.

Positive effects:

- (1) Smaller page table - fewer pages per process means smaller page table, less memory overhead for page tables.
- (2) Higher TLB hit rate - more data/instructions fit per page, better locality, fewer TLB misses.

Negative effects:

- (1) More internal fragmentation - last page of each process is mostly empty, wasting memory.
- (2) Longer page fault service time - more data must be read from disk per page fault.

[CHN] 优点: (1) 页表更小——每进程页数减少, 页表内存开销更低。

(2) TLB命中率更高——每页容纳更多数据/指令, 局部性更好。

缺点: (1) 内部碎片增加——每进程最后一页大部分为空。

(2) 缺页中断处理时间更长——每次缺页需从磁盘读更多数据。

Q: [4 marks] 3. Explain base and limit registers for memory protection.

Base register = smallest legal physical address a process can access.

Limit register = range of addresses (size of the allocated region).

Every memory access is checked: if the address < base OR address >= base + limit, a trap to the operating system occurs (protection fault). This ensures one process cannot access another process's memory.

[CHN] 基址寄存器存放进程可访问的最小合法物理地址。

界限寄存器存放地址范围 (已分配区域大小)。

每次内存访问都要检查: 若地址 < 基址 或 地址 >= 基址+界限, 则触发陷入操作系统 (保护错误)。

这确保了一个进程不能访问另一个进程的内存。

Q: [12 marks] 4. Design a web browser using multiple processes and threads.

Use BOTH multiple processes AND multiple threads:

Multiple processes (Chrome approach):

- Each browser tab runs in its own process
- Advantages: (1) One tab crash doesn't affect other tabs. (2) Each process has separate address space for security/sandboxing.

Multiple threads within each process:

- Rendering thread, JavaScript execution thread, I/O thread, UI thread
- Advantages: Threads within same process share resources efficiently, lighter context switches.

This hybrid design gives the best combination of stability, security, and performance.

[CHN] 使用多进程+多线程:

多进程 (Chrome架构):

- 每个标签页运行在独立进程中
- 优点: (1) 一个标签页崩溃不影响其他标签页。 (2) 每个进程有独立地址空间, 实现安全沙箱。

多线程 (每个进程内):

- 渲染线程、JavaScript执行线程、I/O线程、UI线程
- 优点: 同进程内线程共享资源, 切换开销更小。

这种混合设计在稳定性、安全性和性能之间取得最佳平衡。

Q: [12 marks] 5. Discuss advantages and disadvantages of RAID.

Advantages:

- (1) Improved reliability via redundancy - parity (RAID 5/6) or mirroring (RAID 1) protects against disk failures.
- (2) Improved performance via parallelism - striping (RAID 0/5/6) allows concurrent read/write across multiple disks.

Disadvantages:

- (1) Higher cost - more disks needed for redundancy (e.g., RAID 1 needs double the disks).
- (2) Complexity of recovery and management - RAID configuration, reconstruction after failure, and hot spares add administrative overhead.

[CHN] 优点:

- (1) 通过冗余提高可靠性——奇偶校验(RAID 5/6)或镜像(RAID 1)防止磁盘故障。

(2) 通过并行提高性能——条带化(RAID 0/5/6)允许跨多个磁盘并发读写。

缺点:

- (1) 成本更高——冗余需要更多磁盘 (如RAID 1需要双倍磁盘) 。
- (2) 恢复和管理更复杂——RAID配置、故障后重建和热备增加了管理开销。

Question 2 (32 marks)

Q: [8 marks] 1. EDF scheduling: P1(arr=0,t=10,d=33), P2(arr=4,t=3,d=28), P3(arr=5,t=10,d=29).

EDF (Earliest Deadline First) schedules by the earliest deadline.

Timeline:

- t=0: P1 runs (deadline=33, earliest)
- t=4: P2 arrives (deadline=28 < 33), preempt P1, P2 runs
- t=5: P3 arrives (deadline=29), P2 has earlier deadline, P2 continues
- t=7: P2 finishes. Compare P1(d=33) vs P3(d=29). P3 runs (earlier deadline)
- t=17: P3 finishes. P1 resumes, runs until t=27

CPU Utilization: $U = 10/33 + 3/28 + 10/29 = 0.303 + 0.107 + 0.345 = 0.755$

Since $U \leq 1.0$, all tasks can theoretically be scheduled.

[CHN] EDF (最早截止时间优先) 按最早截止时间调度。

时间线:

- t=0: P1运行(截止时间=33)
- t=4: P2到达(截止时间=28<33), 抢占P1, P2运行
- t=5: P3到达(截止时间=29), P2截止更早, P2继续
- t=7: P2完成。比较P1(d=33)与P3(d=29), P3运行
- t=17: P3完成。P1恢复运行至t=27

CPU利用率 $U = 0.755$, 理论上可调度。

Q: [6 marks] 2. FIFO page replacement: ref=1 2 3 2 1 5 2 1 6 2 5 6 3 1 3 6 1 2 4 3, frames=3.

FIFO (First-In-First-Out) replaces the oldest page in frames.

Step-by-step trace (frames=3):

1: [1,-,-] FAULT | 2: [1,2,-] FAULT | 3: [1,2,3] FAULT
2: HIT | 1: HIT | 5: [2,3,5] FAULT
2: HIT | 1: [3,5,1] FAULT | 6: [5,1,6] FAULT
2: [1,6,2] FAULT | 5: [6,2,5] FAULT | 6: HIT
3: [2,5,3] FAULT | 1: [5,3,1] FAULT | 3: HIT
6: [3,1,6] FAULT | 1: HIT | 2: [1,6,2] FAULT
4: [6,2,4] FAULT | 3: [2,4,3] FAULT

Total page faults: 15

[CHN] FIFO (先进先出) 替换最旧的页面。

逐步追踪 (3帧) :

1:缺页 [1,-,-] | 2:缺页 [1,2,-] | 3:缺页 [1,2,3]
2:命中 | 1:命中 | 5:缺页 [2,3,5]
2:命中 | 1:缺页 [3,5,1] | 6:缺页 [5,1,6]

2:缺页 [1,6,2] | 5:缺页 [6,2,5] | 6:命中
3:缺页 [2,5,3] | 1:缺页 [5,3,1] | 3:命中
6:缺页 [3,1,6] | 1:命中 | 2:缺页 [1,6,2]
4:缺页 [6,2,4] | 3:缺页 [2,4,3]

总缺页次数: 15

Q: [6 marks] 3. SSTF disk scheduling: requests=44,20,95,4,50,52,47,61,87,25, head=50.

SSTF (Shortest Seek Time First) always services the nearest request.

Head starts at 50.

Service order: 50 -> 52 -> 47 -> 44 -> 25 -> 20 -> 4 -> 61 -> 87 -> 95

Movement calculation:

$|50-52|=2$, $|52-47|=5$, $|47-44|=3$, $|44-25|=19$, $|25-20|=5$, $|20-4|=16$, $|4-61|=57$, $|61-87|=26$, $|87-95|=8$

Total head movement = $2+5+3+19+5+16+57+26+8 = 141$ cylinders

[CHN] SSTF (最短寻道时间优先) 总是服务最近的请求。

磁头从50开始。

服务顺序: 50 -> 52 -> 47 -> 44 -> 25 -> 20 -> 4 -> 61 -> 87 -> 95

移动量计算:

$2+5+3+19+5+16+57+26+8 = 141$

总磁头移动量 = 141柱面

Q: [6 marks] 4. TLB Effective Access Time: TLB=20ns, memory=50ns, hit rate=70%.

TLB hit path: TLB lookup + memory access = $20 + 50 = 70$ ns

TLB miss path: TLB lookup + memory access (for page table) + memory access (for data) = $20 + 50 + 50 = 120$ ns

$EAT = hit_rate * hit_time + miss_rate * miss_time$

$EAT = 0.7 * 70 + 0.3 * 120$

$EAT = 49 + 36$

$EAT = 85$ ns

Effective Access Time = 85 nanoseconds.

[CHN] TLB命中路径: TLB查找+内存访问 = $20+50 = 70$ ns

TLB未命中路径: TLB查找+页表内存访问+数据内存访问 = $20+50+50 = 120$ ns

$EAT = 0.7 * 70 + 0.3 * 120 = 49 + 36 = 85$ ns

有效访问时间 = 85纳秒。

Q: [6 marks] 5. Segmentation: seg0(300,4000), seg1(600,1000), seg2(500,2700), seg3(900,1800), seg4(1000,2500). Calculate physical addresses.

Segment table: (limit, base)

seg0(300, 4000), seg1(600, 1000), seg2(500, 2700), seg3(900, 1800), seg4(1000, 2500)

(1) seg1, offset 550: Check $550 < 600$ (limit)? YES. Physical = $1000 + 550 = 1550$

(2) seg3, offset 908: Check $908 < 900$ (limit)? NO ($908 > 900$). TRAP - illegal address, segmentation fault.

(3) seg4, offset 670: Check $670 < 1000$ (limit)? YES. Physical = $2500 + 670 = 3170$

[CHN] 段表: (界限, 基址)

seg0(300,4000), seg1(600,1000), seg2(500,2700), seg3(900,1800), seg4(1000,2500)

(1) seg1, 偏移550: $550 < 600$? 是。物理地址 = $1000 + 550 = 1550$

(2) seg3, 偏移908: $908 < 900$? 否($908 > 900$)。陷入——非法地址, 段错误。

(3) seg4, 偏移670: $670 < 1000$? 是。物理地址 = $2500 + 670 = 3170$

Question 3 (12 marks) - Banker's Algorithm

Q: [12 marks] Banker's Algorithm: Determine if the system is in a safe state and whether P2's request (0,5,0) can be granted.

Given matrices (resources R1, R2, R3):

Total: $R1=1, R2=5, R3=2$

Need = Max - Allocation:

P1: (0,0,0) P2: (0,7,5) P3: (1,0,0) P4: (0,0,2)

Available = (0, 4, 5)

Safety check:

1. P1 needs (0,0,0) - Available covers it. P1 finishes. Available = (0,4,5)
2. P4 needs (0,0,2) - Available (0,4,5) $\geq$ (0,0,2). P4 finishes. Available = (0,10,8)
3. P2 needs (0,7,5) - Available (0,10,8) $\geq$ (0,7,5). P2 finishes. Available = (1,10,13)
4. P3 needs (1,0,0) - Available (1,10,13) $\geq$ (1,0,0). P3 finishes.

Safe sequence: P1 -> P4 -> P2 -> P3. System is SAFE.

For P2's request (0,5,0):

- Check if Request $\leq$ Available: (0,5,0) $\leq$ (0,4,5)? NO ($R2: 5 > 4$). Request CANNOT be granted immediately. P2 must wait.

[CHN] 银行家算法:

Need = Max - Allocation:

P1:(0,0,0) P2:(0,7,5) P3:(1,0,0) P4:(0,0,2)

Available = (0, 4, 5)

安全检查:

1. P1需要(0,0,0), 可满足, 完成后 Available=(0,4,5)
2. P4需要(0,0,2), 可满足, 完成后 Available=(0,10,8)
3. P2需要(0,7,5), 可满足, 完成后 Available=(1,10,13)
4. P3需要(1,0,0), 可满足, 完成

安全序列: P1->P4->P2->P3。系统安全。

P2请求(0,5,0): 检查 Request $\leq$ Available? (0,5,0) $\leq$ (0,4,5)? 否($R2: 5 > 4$)。

请求不能立即批准, P2必须等待。

Question 4 (12 marks) - Barbershop Problem

Q: 1. Explain the initial values of semaphores: customers=0, barbers=0, mutex=1.

customers = 0: Counting semaphore representing the number of waiting customers. Initially 0 means no customers are waiting.

barbers = 0: Counting semaphore representing the number of idle barbers. Initially 0 means the barber is sleeping (not ready).

mutex = 1: Binary semaphore (mutex lock) protecting the 'waiting' variable (critical section). Initially 1 means the critical section is free.

[CHN] customers=0: 计数信号量, 表示等待顾客数。初始为0表示没有顾客等待。

barbers=0: 计数信号量, 表示空闲理发师数。初始为0表示理发师在睡觉。

mutex=1: 二进制信号量 (互斥锁), 保护'waiting'变量 (临界区)。初始为1表示临界区空闲。

Q: 2. What happens if waiting++ and waiting-- are moved after V(mutex)?

This creates a RACE CONDITION. If the increment/decrement of 'waiting' happens after releasing the mutex:

- Two customers could both read the same 'waiting' value
- Both increment it (e.g., both read 0, both write 1)
- The actual count becomes incorrect
- This could lead to customers being turned away when chairs are available, or more customers admitted than chairs exist

The critical section MUST include the read-modify-write of 'waiting' to ensure atomicity.

[CHN] 这会产生竞态条件。如果waiting++和waiting--在释放互斥锁之后执行:

- 两个顾客可能同时读取相同的waiting值
- 两者都递增 (例如都读到0, 都写入1)
- 实际计数变得不正确
- 可能导致有椅子时顾客被拒, 或进入的顾客超过椅子数

临界区必须包含waiting的读-改-写操作以确保原子性。

Q: 3. What happens when CHAIRS = 0?

When CHAIRS = 0, there are no waiting chairs in the barbershop.

Customer behavior:

- Customer arrives, enters critical section
- Checks: waiting >= CHAIRS (0)?
- If waiting >= 0 (always true after one customer), customer LEAVES immediately
- Essentially: only 1 customer can be served at a time, with NO queue

Barbershop becomes a 'serve one at a time' system:

- If barber is busy, customer leaves and must come back later
- No waiting = no queue
- This is like a single-server system with no buffer

[CHN] 当CHAIRS=0时, 理发店没有等待椅。

顾客行为:

- 顾客到达, 进入临界区
- 检查: waiting >= CHAIRS(0)?

- 如果 $\text{waiting} \geq 0$ (有顾客后总为真), 顾客立即离开
- 本质上: 一次只能服务一个顾客, 没有排队

理发店变成'逐个服务'系统:

- 如果理发师忙, 顾客离开, 稍后再来
- 没有等待 = 没有队列
- 类似无缓冲的单服务器系统